

Google 기반 바이브 코딩 실습 최소 가이드

송경모

2026-04-15

1. 바이브 코딩의 개요 및 서비스 현황

바이브 코딩(Vibe Coding)은 개발자가 코드 한 줄 한 줄을 직접 타이핑하는 대신, 대규모 언어 모델(LLM)과 자연어로 소통하며 시스템의 '느낌(Vibe)'과 '구조'를 설계하여 소프트웨어를 빌드하는 새로운 방식입니다.

1.1 Google 기반 바이브 코딩 플랫폼

구글은 Gemini 3 시리즈를 기반으로 강력한 에이전트 기능을 활용한 바이브 코딩 환경을 제공합니다. Gemini 기반의 바이브 코딩은 주로 다음 세 가지 환경에서 이루어집니다.

- **Google AI Studio (Build in AI Studio):** 가장 대표적인 바이브 코딩 플랫폼입니다. 아이디어를 설명하면 AI가 코드와 파일을 생성하고, 라이브 프리뷰(Live Preview)를 통해 즉시 앱 실행해 볼 수 있습니다. (예: “스타트업 이름 생성기 만들어줘”라고 입력하면 즉시 웹 앱이 빌드됨)
- **Gemini Code Assist (IDE 연동):** VS Code나 JetBrains 같은 전문 편집기에 설치하여 사용합니다. 기존 코드를 이해하고 대화를 통해 기능을 수정하거나 전체 파일을 변환하는 방식의 바이브 코딩을 지원합니다.
- **Google Cloud Shell Editor:** 별도의 설정 없이 웹 브라우저에서 바로 사용할 수 있는 개발 환경으로, Gemini Code Assist가 사전 탑재되어 있어 50시간 무료 이용이 가능합니다.

서비스명	대상 및 접근 수준	과금 및 특징
Google AI Studio	초급 (웹 브라우저 접근)	무료(제한적) / Pro tier 유료. Live Preview로 즉시 앱 빌드.
Gemini Code Assist	중급 (VS Code/IDE 플러그인)	개인 무료 tier 제공 / 기업용 월 \$19~\$45. 전문 개발 환경 통합.
Cloud Shell Editor	중급 (브라우저 내 IDE)	50시간 무료 제공. 설치 없이 터미널과 에디터 동시 사용 가능.

구글의 Gemini 3 시리즈는 강력한 에이전트 기능(Agentic capabilities)과 바이브 코딩(Vibe Coding) 능력을 갖추고 있어, 개발자가 코드 한 줄 한 줄을 직접 작성하는 대신 AI와 대화하며 애플리케이션을 빌드하고 배포하는 환경을 제공합니다.

2. 유료 및 무료 서비스 구분

Gemini의 코딩 서비스는 개인 사용자부터 대규모 기업까지 층위별로 나뉩니다.

무료 tier (Free Tier)

- 대상: 개인 개발자, 학생, 취미 개발자.
- 포함 사항:
 - **Gemini Code Assist for individuals:** VS Code, JetBrains 플러그인 무료 사용.

- **Gemini CLI:** 터미널 기반의 바이브 코딩 워크플로우 지원.
- **AI Studio:** 최신 모델(Gemini 3.1 Pro/Flash 등)을 통한 테스트 및 앱 프로토타이핑.
- **제한:** 일일 요청 횟수(Quotas) 및 분당 호출 수(RPM) 제한이 엄격하며, 데이터가 모델 학습에 활용될 수 있습니다.

유료/구독 티어 (Paid Tier)

유료 서비스는 개인용 구독과 기업용 라이선스로 나뉩니다.

구분	서비스명	주요 혜택
개인 프리미엄	Google AI Plus / Ultra	Gemini 3.1 Pro 등 가장 성능이 뛰어난 모델에 대한 우선 접근권, 훨씬 높은 사용 할당량(Quotas) 제공.
기업용 (Standard)	Gemini Code Assist Standard	월 \$19부터 시작. 구글 클라우드 환경과 통합되며, 기업의 보안 정책을 준수(데이터 학습 제외).
기업용 (Enterprise)	Gemini Code Assist Enterprise	월 \$45부터 시작. 대규모 코드베이스 커스터마이징, 고급 보안 기능, 대규모 조직 관리 기능 포함.

3. 요약: 바이브 코딩을 시작하려면?

바이브 코딩의 핵심은 “설명하고(Describe) → 생성하고(Generate) → 실행하고(Execute) → 정교화(Refine)”하는 루프입니다.

- **초보자:** **Google AI Studio**의 ‘Build’ 기능을 통해 코딩 없이 앱을 만들어보는 것을 추천합니다.
- **숙련자:** **Gemini Code Assist**를 IDE에 설치하여 본인의 프로젝트를 AI와 함께 대화하며 고도화하는 방식을 추천합니다.

Gemini 3 모델은 특히 복잡한 문제 해결과 에이전트 성능이 강화되어, 과거보다 훨씬 정교한 바이브 코딩이 가능해졌습니다.

2. 접근 절차 및 준비 사항

바이브 코딩 작업에 들어가기 전, 도구의 설정과 과업의 정의를 명확히 하는 것이 성공의 핵심입니다.

2.1 단계별 접근 절차

1. **인터페이스 선택:** 웹 기반(Google AI Studio, OpenAI ChatGPT)으로 시작하여 점차 IDE(Cursor, VS Code)로 확장합니다.
2. **필수 SW 및 플러그인 설치:**
 - **Cursor:** AI 네이티브 에디터로, 바이브 코딩에 가장 최적화된 도구입니다.
 - **Gemini Extension:** VS Code 내에서 구글의 코딩 어시스턴트를 호출하기 위해 필요합니다.
3. **과금 수준 확인:** 무료 티어의 할당량(Quota)을 확인하고, 대규모 프로젝트 시 API 호출 비용을 산정합니다.

2.2 과업 확정 방법 (Definition of Task)

AI에게 작업을 맡기기 전 다음 요소를 확정해야 합니다.

- **프로그래밍 환경**: 사용할 언어(Python, JavaScript 등)와 프레임워크(Streamlit, React 등)를 명시합니다.
- **목적 및 구조 설계**: 앱이 해결해야 할 문제와 데이터의 흐름(Input -> Process -> Output)을 정의합니다.

3. 작업 범위와 수준 및 한계

- **수준**: 단순한 스크립트 작성부터 풀스택 웹 애플리케이션 프로토타이핑까지 가능합니다.
- **범위**: UI 디자인, 데이터베이스 연동, 비즈니스 로직 구현을 포괄합니다.
- **한계**: AI는 매우 복잡한 대규모 아키텍처에서 논리적 일관성을 잃을 수 있으며, 보안 취약점이 포함된 코드를 생성할 가능성이 있습니다.

4. 바이브 코딩 작업 절차 안내

4.1 프롬프트 작성 원칙 및 예시

가장 간단하게로는 대화창에 프롬프트 입력만으로 가능합니다. 이때 막연한 요청으로 끝나지 말고 구체적인 페르소나와 제약 조건을 제공해야 합니다.

- **예시**: “너는 Python Streamlit 전문가야. 사용자가 업로드한 CSV 파일에서 결측치를 찾아 시각화해주는 웹 앱을 만들어줘. 디자인은 다크 모드로 설정해.”

4.2 설계용 .md 파일 구성 및 작성 원칙

더 나아가 AI에게 복잡한 과업을 전달할 때는 별도의 .md 파일을 첨부하는 것이 유리합니다.

- **원칙**: 선언적(Declarative)으로 작성하되, 모호한 표현을 피하고 기술 스택을 명시합니다. 다음 내용을 적절한 이름의 .md파일에 저장하고 이 파일을 대화창에 첨부한 뒤 “이 설계도대로 프로젝트 환경을 구축하고 코드를 작성해 줘” 라는 식으로 작업을 요청합니다.

```
# Project: AI 가계부 앱
## 1. 기능 요구사항
- 영수증 사진을 업로드하면 OCR로 금액 추출
- 월별 지출 통계 그래프 출력
## 2. 기술 스택
- 언어: Python 3.10+
- 라이브러리: Pandas, Plotly, OpenAI API
## 3. UI 구성
- 사이드바: 날짜 필터링
- 메인화면: 지출 요약 대시보드
```

위에 제시된 기능 요구사항, 기술 스택, UI 구성을 그대로 프롬프트로 사용하면, AI는 가장 일반적이고 단순한 형태의 코드를 작성합니다.

기술 스택이나 UI 구성에 포함될 내용에 대하여 사전 지식이 없다면, 미리 이런 내용들을, 예를 들어 “어떤 Python 라이브러리를 사용하는 것이 필요한가?” 등을 AI chat 창에서 질문하면서 어느 정도 사전 지식을 갖춘 다음에 이를 프롬프트에 반영합니다.

하지만 ‘바이브 코딩’의 완성도를 평균적인 수준을 넘어 자신의 필요에 맞추어 더욱 높이기 위해서는, 프로젝트의 성격에 따라 세부 맥락이 추가되어야 합니다.

예를 들어 이 가계부 앱의 경우에는 다음과 같은 내용들에 대하여 User가 지침을 결정해서 추가될 수 있습니다.

- 데이터 처리의 구체성: “OCR로 추출한 데이터를 어떤 형식(JSON 등)으로 저장할 것인가?” (예: 로컬 SQLite DB 사용 여부)
- 에러 핸들링: “영수증 사진이 흐릿하여 인식이 안 될 경우 사용자에게 어떤 피드백을 줄 것인가?”
- 라이브러리 버전 관리: “최신 OpenAI SDK(v1.0+) 버전을 사용할 것인가?”
- 결과물의 형태: “단일 파일(app.py)로 만들 것인가, 아니면 기능별로 파일을 분리할 것인가?”

[학생 입장에서 현실적인 프롬프트 입력 과정]

1. 반복적, 대화형 프롬프트 입력

학생들은 물론이고, 현업 개발자들도조차도 위 요구사항을 * ‘기본 뼈대’ 로 삼되, AI와 대화를 주고받으며(Iterative process) 살을 붙여나가는 과정이 필요합니다.

프로젝트의 모든 세부 작동 과정을 머릿 속에 그릴 수 있는 사전 업역 지식(domain knowledge)이 없는 한, 첫 프롬프트는 대개 중요한 구체사항들이 누락될 수 있습니다. 현실적으로는, 학생이 구상할 수 있는 가능한 범위 내에서 첫 프롬프트로 작업을 시작하고, 그 뒤 이 앱을 실행하는 과정에서 계속해서 부족한 점이나 에러가 등장하게 되면, 그때마다 AI와 계속 대화하면서 부족한 점을 계속 채워넣고 에러가 발생하는 부분을 수정해 나가는 전략을 취할 필요가 있습니다.

2. 프롬프트 구문 자체를 AI에 요구

많은 학생들이, 예를 들어 당장 “학습 진도 관리 앱”을 만들고 싶어 해도 막상, 프롬프트를 어떻게 입력해야 할지, 구체적인 지시 사항을 어떻게 서술해야 할지, 막막하게 느낄 수 있습니다. 이럴 때에는 “나는 대학생이고 AI 바이트 코딩 초보자야. 나는 학기 중 수강하는 과목들에 대해 학습 진도를 관리하는 앱을 만들고 싶어. 그런데 나는 프롬프트를 어떻게 작성해야 할지 모르는 상태이니, 적절한 프롬프트 문안을 작성해줘.” 로 시작해서 여러 차례 대화를 통해 프롬프트 문안을 완성하는 것도 좋은 방법입니다.

가계부 앱 외에 프로젝트 예시를 두 가지만 더 들면 다음과 같습니다.

예시 A: AI 기반 개인 맞춤형 운동 루틴 생성기

단순한 운동 목록 나열이 아니라, 사용자의 체격 조건과 목표를 분석하여 시각화된 계획을 제공하는 프로젝트입니다.

- 1. 기능 요구사항:
 - 사용자의 키, 몸무게, 운동 목적(다이어트/근비대) 입력 폼.
 - LLM을 활용하여 주간 운동 스케줄 및 식단 가이드 생성.
 - 운동 부위별 비중을 파이 차트(Pie Chart)로 시각화.
- 2. 기술 스택:
 - 언어: Python 3.10+
 - 프레임워크: Streamlit (빠른 UI 구현)
 - 시각화: Matplotlib 또는 Seaborn
- 3. UI 구성:
 - 입력창: 슬라이더와 선택 박스로 구성된 신체 정보 입력.
 - 결과창: 탭(Tab) 구성을 통해 ‘운동 계획’ 과 ‘영양 가이드’ 분리 표시.

예시 B: 뉴스 요약 및 감성 분석 대시보드

특정 키워드를 검색하면 최신 뉴스를 가져와 요약하고, 해당 뉴스가 긍정적인지 부정적인지 분석해 주는 프로젝트입니다.

- 1. 기능 요구사항:
 - 특정 키워드(예: ‘생성형 AI’) 검색 시 관련 뉴스 크롤링 또는 API 호출.
 - 각 뉴스 기사의 핵심 내용을 3줄로 요약.

- 뉴스 전체의 감성 상태(Sentiment)를 점수화하여 게이지 차트로 표시.
- 2. 기술 스택:
 - 언어: Python
 - 도구: BeautifulSoup (크롤링) 또는 NewsAPI
 - 분석: OpenAI GPT-4o (요약 및 감성 분석)
- 3. UI 구성:
 - 상단: 키워드 입력창과 검색 버튼.
 - 중단: 주요 뉴스 카드 리스트.
 - 하단: 전체 뉴스 흐름을 보여주는 감성 지수 그래프.

4.3 디버깅 및 배포 유의 사항

- 디버깅: 처음 AI가 생성해준 코드를 실행하다보면 에러 메시지를 만날 확률이 높습니다. 이 에러 메시지를, 텍스트 Ctrl+C 하거나 화면캡처한 이미지(이미지는 텍스트보다 토큰을 더 많이 사용함에 유의!)를 그대로 AI에게 복사하여 붙여넣고 원인 분석과 해결책을 요청합니다. “[에러 첨부 후] 원인을 분석하고 해결책을 제시해줘!”
- 배포: API 키와 같은 민감 정보는 반드시 .env 파일에 격리하고, 배포 전 AI가 생성한 코드의 라이선스를 확인합니다. 학생이 과제 형태로 작성한 앱은 학생이 개인 용도로만 사용하면 큰 문제가 없다. 그러나 이를 제3자에게 공유하거나 상업용으로 활용하려고 한다면 라이선스, 네트워크 설정, 보안 설정, 과금 설계 등에서 차원이 다른 매우 복잡한 이슈가 등장합니다. 이런 내용에 대해서는 다시 AI에 앱에 대한 정보를 다양하게 제시한 뒤, 어떤 조치를 취해야 할지 반드시 사전 점검하고 이르 반영한 뒤 배포하여야 합니다.

5. 최상위 단계: Harness Engineering (하네스 엔지니어링)

하네스 엔지니어링은 AI가 단순히 코드를 생성하는 것에서 나아가, 그 코드가 올바른지 (사람 개입 없이 AI가) 스스로 검증하고 수정하게 만드는 자동화된 환경을 구축하는 것입니다. 이 작업은 대규모 상업용 또는 기업 프로젝트를 실행하는 전문 개발자 직군에서 수행하는 것이 관행이나, 개인 용도의 앱을 제작하는 경우에도 충분히 적용할 수 있다. 하네스 엔지니어링에 대한 상세한 지침, 그리고 그 장단점과 유의사항에 대해서는 별도의 지식 영역이므로, AI에 세부 질문 또는 전문 지침서를 통해 충분히 학습하고 임할 것을 권장합니다.

1. 테스트 하네스 설계: AI에게 코드와 함께 해당 코드를 검증할 ‘유닛 테스트(Unit Test)’ 코드를 동시에 작성하게 합니다.
2. 자기 수정 루프(Self-Correction Loop): 테스트 코드 실행 결과(에러 로그)를 다시 AI에게 피드백으로 주어, AI가 스스로 코드를 고치게 만드는 반복 프로세스를 설정합니다.
3. 지속적 통합: 검증된 모듈을 기존 시스템에 안전하게 결합하는 파이프라인을 구축하여 에이전트 중심의 개발을 완성합니다.